


# Learn Low-Level Design in 25 Days: Day 18— Thread Safety and Concurrency in LLD

Ever pushed a feature to production that worked flawlessly in testing—only to crash spectacularly when multiple users hit it at once?

**TechPreneur**

Published October 30, 2025

Free: No

Read on Freedium · original

# Contents


Why Thread Safety Matters 0

Core Concepts: Concurrency 101 0

1. Threads and Shared State 0

2. Synchronization 0

3. Lock Granularity 0

4. Immutable Objects 0

5. Thread-Safe Collections and Patterns 0

Case Study: Designing a Thread-Safe Cache 0

Problem 0

Solution 0

Visual Explanation 0

Try It Yourself 0

Summary 0

Up Next 0

Maybe it was a shared counter, a cached object, or a background task that started stepping on another thread's toes. You might've stared at your logs wondering:

> "How on earth can this variable be negative? I literally just checked it."

Welcome to the chaotic world of **concurrency**.

Today is **Day 18** of the Learn Low-Level Design in 25 Days series — and we're diving into a concept every system designer must master: **thread safety and concurrency**.




By the end of this post, you'll understand:

- Why concurrent systems are tricky
- How to design thread-safe classes
- What patterns and principles prevent race conditions
- And how companies like Uber, WhatsApp, and Instagram handle multi-threaded operations at scale

Save and follow the full series here. (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)

## Why Thread Safety Matters

Imagine you're designing a **Ride Matching Service** for Uber.

Every time a rider requests a trip, the system finds an available driver and marks them as "busy." But multiple requests can come in **simultaneously**, and two threads might both see the same driver as "available" — assigning them to two different riders.

That's a race condition.

It's not a code bug in logic — it's a **timing bug**. Two correct pieces of code collide when executed concurrently.

Concurrency issues like this are silent killers in production systems — they may only occur 1% of the time, but that 1% can ruin your data integrity or cause deadlocks that bring your service to its knees.

## Core Concepts: Concurrency 101

## 1. Threads and Shared State

A **thread** is a lightweight execution unit. Multiple threads can run in parallel to speed up operations — but when they share mutable data, chaos can ensue.

Example problem:

```
class Counter {
    private int count = 0;
    public void increment() {
        count++; // Not thread-safe!
    }
    public int getCount() {
        return count;
    }
}
```

If two threads call `increment()` at the same time, they might both read the same `count`, increment it, and write back — resulting in **lost updates**.

## 2. Synchronization

We use synchronization to ensure **only one thread** modifies a shared resource at a time.

```
class SafeCounter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public synchronized int getCount() {
        return count;
    }
}
```

This is **thread-safe**, but note the trade-off — locking reduces concurrency and can become a bottleneck.

## 3. Lock Granularity

Good designers balance **safety** and **performance**.

- **Coarse-grained locks** — simpler, but block too much work.
- **Fine-grained locks** — more concurrency, but more complexity.

Example: a `ConcurrentHashMap` in Java uses **segmented locks**, allowing multiple threads to operate on different buckets simultaneously.

## 4. Immutable Objects

The easiest way to achieve thread safety? **Make data immutable.**

Immutable objects can be freely shared between threads without synchronization.

Example:

```
record Point(int x, int y) {} // Java 16+
```

Since a `Point` can't change once created, no thread can corrupt it. Functional languages and modern Java/Kotlin code often prefer immutability for this reason.

## 5. Thread-Safe Collections and Patterns

Real-world concurrent systems rely on well-tested patterns and utilities:

- **Thread-safe collections**: `ConcurrentHashMap`, `CopyOnWriteArrayList`
- **Atomic operations**: `AtomicInteger`, `AtomicReference`

- **Producer-Consumer pattern** using `BlockingQueue`
- **Executor Services** for managing thread pools
- **Double-Checked Locking** for lazy initialization

## Case Study: Designing a Thread-Safe Cache

Let's revisit our **cache system** from Day 17. We'll now make it **thread-safe** for concurrent reads and writes.

## Problem

Multiple threads may read/write cache entries simultaneously:

```
cache.put("user:123", user);
cache.get("user:123");
```

Without synchronization, one thread might read half-updated data, or two threads may write conflicting values.

## Solution

Use a **ConcurrentHashMap** with atomic operations:

```
class ThreadSafeCache<K, V> {
    private final ConcurrentHashMap<K, V> cache = new ConcurrentHashMap<>();

    public V get(K key) {
        return cache.get(key);
    }

    public void put(K key, V value) {
        cache.put(key, value);
    }

    public V getOrLoad(K key, Supplier<V> loader) {
        return cache.computeIfAbsent(key, k -> loader.get());
    }
}
```

This design ensures:

- Concurrent reads/writes don't corrupt the map.
- Loading is atomic — only one thread calls the `loader` for a missing key.

That's how modern systems like **Netflix's Hystrix**, **Spring Cache**, or **Guava Cache** maintain high concurrency safely.

## Visual Explanation

Here's a simple illustration of locking:

```
Shared Resource
      |
   +--------+
   |  Lock  |
   +--------+
   /        \
Thread 1   Thread 2
 acquire     waits
   ↓          ↓
 perform     acquire
 release     perform
```

And a conceptual view of a thread-safe cache:

```
+----------------------+
| ThreadSafeCache<K,V> |
|----------------------|
| +ConcurrentHashMap   |
| +get()               |
| +put()               |
| +computeIfAbsent()   |
+----------------------+
         |
  +---------------+
  | Thread Pool   |
  +---------------+
```

## Try It Yourself

1. **Practice Task 1:** Implement a thread-safe "Bank Account" class that supports deposit and withdrawal operations from multiple threads.
2. **Practice Task 2:** Design a mini "OrderProcessor" that uses a `BlockingQueue` to process orders asynchronously — producer adds orders, consumer processes them.

Reflect:

- Which variables in your current project might be accessed by multiple threads?
- Can they be made immutable or protected via locks?

## Summary

Let's recap today's key takeaways:

- **Thread safety** ensures predictable behavior when multiple threads access shared data.
- Use **locks, immutability, and concurrent collections** wisely.
- Avoid over-synchronization — it can hurt performance.
- Always test concurrent code under stress (tools like JMeter or thread simulators help).
- Patterns like **Producer-Consumer**, **Thread Pool**, and **Immutable Objects** are your concurrency allies.

## Up Next

Tomorrow, we'll put concurrency to the test in a real-world system.

**Day 19 — Designing a Food Delivery App (Swiggy/UberEats Style)** We'll integrate caching, notification queues, and concurrency handling — all in one design.

Follow and save the series here. (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)